

Estudo Dirigido 03

Padronização em Banco de Dados

Disciplina: Administração de Banco de Dados

Professor: Jacimar Tavares

Alunos: Italo Butinholi Mendes, Vitório Costa Ribeiro, Mateus
Cacilhas Freire da Paz, Lucas Nascif e Gustavo Côrrea

1. Introdução

A padronização em sistemas de banco de dados é uma prática indispensável para o ciclo de vida de qualquer software. Ela consiste em definir um conjunto de regras, convenções e boas práticas que guiam desde a modelagem até a manutenção diária dos dados. No contexto do **MySQL**, SGBD escolhido para a aplicação deste estudo, a ausência de padronização pode resultar em anomalias, redundâncias e degradação de desempenho. Por outro lado, um ambiente padronizado garante a longevidade do sistema, facilita a integração com novas ferramentas, simplifica a curva de aprendizado para novos desenvolvedores e assegura a consistência das informações em todo o banco.

Projeto utilizado no trabalho: https://github.com/ItaloBM/EstudoDirigido03_Padroniza-oemBancodeDados

2. Aplicação das Práticas de Padronização no MySQL

Para ilustrar as regras, utilizaremos como base um banco de dados hipotético de um **Sistema de E-commerce e Serviços**.

2.1. Convenções de Nomenclatura

A nomenclatura consistente é a base da comunicação entre a equipe técnica.

- **Tabelas:** Devem ser nomeadas sempre no **singular** e em letras minúsculas `cliente`, `venda`, `produto` (ex: `id_cliente`, `id_produto`).
- **Chaves Primárias (PK):** O prefixo `id_` seguido do nome da tabela (ex: `id_cliente`, `id_produto`).
- **Chaves Estrangeiras (FK):** Devem possuir o exato mesmo nome da Chave Primária da tabela de origem para facilitar os comandos `JOIN` (ex: a tabela `venda` terá a coluna `id_cliente`).

Justificativa: Essa convenção elimina ambiguidades. Um programador saberá imediatamente que `id_cliente` é a chave que liga a venda ao cliente, sem precisar consultar o dicionário de dados o tempo todo.

2.2. Padronização de Tipos de Dados

O uso do tipo de dado correto garante integridade e otimiza o armazenamento no disco e no Buffer do SGBD.

- **Strings Fixas (ex: CPF, CEP):** Utilizar `CHAR` em vez de `VARCHAR`. Ex: `cpf CHAR(11)`.

- **Valores Monetários:** Utilizar `DECIMAL` para evitar problemas de arredondamento. Ex: `DECIMAL(10,2)` `preco_base`
- **Datas:** Utilizar `DATE` (ocupando apenas 3 bytes) quando o horário não importa, e `DATETIME` (8 bytes) quando o registro de hora/minuto for essencial (ex: momento de uma compra).

Justificativa: Evita o desperdício de espaço. Se um campo nunca terá mais de 11 caracteres, usar `VARCHAR(255)` é ineficiente. Além disso, usar tipos numéricos corretos impede a inserção de texto acidental em cálculos matemáticos.

2.3. Integridade dos Dados

Garantir que não haja "dados lixo" ou relacionamentos quebrados.

- **Regra de Entidade:** Toda tabela deve ter uma `PRIMARY KEY` com autoincremento (`AUTO_INCREMENT`).
- **Regra Referencial:** O uso de `FOREIGN KEY` é obrigatório para evitar registros órfãos.
- **Regra de Domínio:** Uso da restrição `CHECK` para impedir valores ilógicos.

Exemplo Prático no MySQL:

```
CREATE TABLE item_venda (
  id_item INT AUTO_INCREMENT PRIMARY KEY,
  id_venda INT NOT NULL,
  quantidade INT NOT NULL,
  FOREIGN KEY (id_venda) REFERENCES venda(id_venda) ON DELETE CASCADE,
  CONSTRAINT chk_qtd CHECK (quantidade > 0)
);
```

2.4. Normalização

A estrutura do banco deve obedecer às Três Primeiras Formas Normais (1FN, 2FN e 3FN) antes de ir para produção.

- **1FN (Valores Atômicos):** Não criar colunas como `telefones_contato` (guardando 2 telefones separados por vírgula). Cria-se uma tabela associativa `telefone_cliente`.
- **2FN (Dependência Total):** Eliminar dependências parciais de chaves compostas.
- **3FN (Dependências Transitivas):** Nenhum campo pode depender de uma coluna que não seja a chave primária.

Justificativa: Normalizar reduz drasticamente a redundância (repetição de dados). Isso impede anomalias de atualização, onde alterar o nome de uma categoria de produto exigiria alterar milhares de linhas, caso não estivessem em uma tabela separada.

2.5. Padrões de Segurança

Segurança deve ser implementada no SGBD, limitando o que os usuários da aplicação podem fazer.

- **Criptografia:** Senhas nunca são guardadas em texto puro, utiliza-se funções de hash nativas como `SHA2(senha, 256)`.
- **Privilégios (DCL):** Criação de perfis (Roles) usando o princípio do privilégio mínimo.

Exemplo Prático:

```
-- Criação de um usuário para a equipe de vendas com permissão apenas de leitura
CREATE USER 'usuario_vendas'@'localhost' IDENTIFIED BY 'senhaForte@123';
GRANT SELECT, INSERT ON ecommerce.venda TO 'usuario_vendas'@'localhost';
REVOKE DELETE, UPDATE ON ecommerce.venda FROM 'usuario_vendas'@'localhost';
```

2.6. Padronização de Índices

Índices aceleram a velocidade de leitura (`SELECT`), mas tornam a gravação (`INSERT` / `UPDATE`) mais lenta.

- **Regra:** Criar índices (além das PKs automáticas) apenas em colunas utilizadas frequentemente em cláusulas `WHERE` (como `cpf`, `email`) e em todas as Chaves Estrangeiras (FKs).

Exemplo Prático:

```
CREATE INDEX idx_email_cliente ON cliente(email);
```

Justificativa: Um índice bem criado evita o Table Scan (varrer a tabela inteira do início ao fim), transformando buscas lentas em respostas instantâneas, o que é vital para a escalabilidade.

2.7. Documentação do Banco de Dados

O banco de dados não deve depender apenas da memória dos desenvolvedores atuais.

- **Dicionário de Dados:** Um documento simples indicando o propósito de cada tabela e o domínio de cada coluna.
- **Comentários no SGBD:** O MySQL permite documentar diretamente na estrutura física usando o parâmetro `COMMENT`.
- **Diagrama (DER):** Manter o modelo conceitual e lógico atualizados e versionados junto ao código-fonte (ex: scripts `.sql` em repositório GitHub).

3. Conclusão

A implantação rigorosa dessas regras de padronização transforma um banco de dados amador em uma infraestrutura profissional. Os benefícios esperados incluem uma redução expressiva de falhas por inconsistência de tipos, maior segurança contra acessos indesejados, agilidade no desenvolvimento (já

que as nomenclaturas tornam o código previsível) e alta escalabilidade com o uso de índices e arquitetura normalizada. Padronizar é garantir que a estrutura corporativa suporte o crescimento do negócio com segurança e integridade.

4. Referências

[1] ABD - Slide de Aula (Módulo 06 - Padronização). Material de Aula.

[2] ELMASRI, R.; NAVATHE, S. *Sistemas de Banco de Dados*. Pearson, 2019.